



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА имени К. А. ТИМИРЯЗЕВА»
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления АПК
Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии
Направленность:
Компьютерные науки и интеллектуальный анализ данных

КУРСОВОЙ ПРОЕКТ

на тему: «Разработка веб-приложения для проведения викторин с
геймификацией».

Выполнил:
Студент 3 курса группы Д-Э-311
Конюхов М.В.

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО подпись

учёная степень, учёное звание, ФИО подпись

учёная степень, учёное звание, ФИО подпись

Оценка _____

Дата защиты _____

Москва 2026

Содержание

Введение	3
Глава 1. Теоретическая часть	5
1.1. Обзор предметной области	5
1.2. Теоретические основы геймификации	6
1.3. Анализ существующих решений	8
1.4. Функциональные требования	10
1.5. Нефункциональные требования	11
1.6. Выбор и обоснование технологического стека	12
Глава 2. Практическая часть	14
2.1. Проектирование базы данных	14
2.2. Структура проекта	15
2.3. Описание реализованного функционала	18
Заключение	22
Список использованных источников	23
Приложения	24

Введение

В связи с быстрым развитием информационных технологий и широким использованием интернета веб-приложения стали главным средством автоматизации учебных и рабочих процессов. Одним из многообещающих направлений является создание систем проверки знаний, в которых используются элементы геймификации. Такой подход повышает заинтересованность пользователей благодаря игровым элементам: начислению баллов, получению уровней, выдаче наград и ведению списков лучших участников.

Актуальность данной работы вызвана растущей потребностью в интерактивных платформах для тестирования знаний в образовании и корпоративном обучении. Обычные способы проверки знаний часто не дают достаточной мотивации участникам, тогда как игровые системы показывают значительно лучшие результаты по удержанию пользователей и качеству усвоения учебного материала.

Цель курсового проекта заключается в создании веб-приложения для проведения викторин с игровыми элементами на основе современного набора технологий.

Для выполнения поставленной цели требуется решить следующие задачи:

- Изучить правила построения информационных систем и способы создания веб-приложений;
- Проанализировать существующие решения в области платформ для тестирования и геймификации;
- Обосновать выбор инструментов и технологий, используемых при разработке;
- Разработать архитектуру системы, схему базы данных и пользовательский интерфейс;
- Создать веб-приложение с системой входа пользователей, модулем викторин и игровыми механиками;
- Проверить работоспособность разработанной системы и проанализировать полученные результаты.

Объектом исследования является процесс проектирования и создания информационных систем на основе веб-технологий.

Предметом исследования выступают методы и средства построения веб-

приложения для викторин с элементами геймификации.

Глава 1. Теоретическая часть

1.1. Обзор предметной области

Веб-викторины представляют собой интерактивные информационные системы, предназначенные для проверки знаний, формирования навыков и повышения вовлечённости пользователей. Такие приложения могут функционировать в двух режимах:

- синхронном, когда несколько участников одновременно выполняют задания под управлением ведущего;
- асинхронном, где пользователь проходит задания в удобное для себя время.

Формат вопросов варьируется от простых вариантов множественного выбора до открытых вопросов и заданий на заполнение пропусков. Выбор формата влияет на способы оценивания и анализа полученных результатов.

В научной литературе геймификация рассматривается как инструмент, который при правильном использовании способен повышать внутреннюю мотивацию пользователей, обеспечивать быструю обратную связь и поддерживать эффект «потока» – то есть оптимальное соотношение между сложностью задания и уровнем навыков участника.

Области применения веб-викторин разнообразны – от образовательных и корпоративных учреждений, до публичных мероприятий и маркетинговых кампаний.

В каждом из этих контекстов требования к функциям системы и показателям эффективности различаются. Для образовательных задач важна точность оценивания и анализ учебных достижений, тогда как для развлекательных мероприятий ключевыми являются вовлечённость аудитории и устойчивый интерес к платформе.

Основные требования предметной области включают:

- поддержку различных типов вопросов;
- надёжный учёт результатов;
- механизмы формирования рейтингов и отчётов;
- возможность масштабируемого проведения сессий в реальном времени.

Также важное значение имеют вопросы безопасности и конфиденциально-

сти данных пользователей, особенно в образовательной и корпоративной среде. Для оценки эффективности систем применяются такие метрики, как вовлечённость, частота повторного использования, доля успешно пройденных заданий и качество обучения. Это позволяет соотнести выбранные игровые механики с реальными учебными или бизнес-целями.

1.2. Теоретические основы геймификации

Геймификация определяется как применение игровых элементов и приёмов в неигровых ситуациях с целью повышения мотивации и вовлечённости пользователей. В основе её эффективности лежат психологические и педагогические теории мотивации. Теория самоопределения выделяет три главные потребности человека:

- автономия – возможность самостоятельно выбирать действия;
- компетентность – ощущение успешности и умения справляться с задачами;
- связанность – чувство принадлежности к группе и взаимодействие с другими.

Удовлетворение этих потребностей способствует росту внутренней мотивации (Ryan & Deci, 2000). Концепция «потока» показывает, что наилучший опыт достигается при равновесии между сложностью задания и уровнем навыков пользователя (Csikszentmihalyi, 1990). Бихевиористские модели делают упор на роль подкрепления: поощрения усиливают желаемое поведение и делают его более частым.

Игровые механики выполняют три взаимосвязанные функции: дают обратную связь, выстраивают структуру прогресса, создают социальное взаимодействие.

В контексте викторин можно воспользоваться следующими элементами:

- очки за правильные ответы;
- бонусы за быстроту реакции;
- бонусы за серии верных ответов;
- уровни пользователей;
- достижения (награды).

Такие механики помогут пользователю ощущать прогресс и рост своей ком-

петентности, побудят возвращаться к заданиям и повысят активность во время прохождения викторины. Например, система очков и бонусов за быстрый ответ закрепляет оперативность и быстроту мышления, тогда как уровни и достижения поддерживают долгосрочную мотивацию, показывая шаги учебного или профессионального развития.

При разработке игровых механик важно учитывать их влияние на обучение. Механики должны усиливать учебную цель, а не заменять её. Очки и таблицы лидеров могут повышать вовлечённость, но если делать слишком большой упор на соревновательность, есть риск, что пользователи будут хуже усваивать материал. Серии правильных ответов и ежедневные задания помогают выработать привычку и регулярно заниматься, что положительно сказывается на запоминании информации. Одновременно необходимо давать обратную связь об ошибках: краткое пояснение после ответа позволяет превратить поведенческую реакцию в осознанное знание.

Правила геймификации должны быть чёткими и понятными. Пользователю необходимо ясно понимать:

- за что начисляются очки;
- как рассчитываются бонусы;
- какие условия нужно выполнить для получения достижения или повышения уровня.

Баланс наград является очень важным условием. Если давать слишком много очков за простые действия, ценность достижений снижается и мотивация падает. Если награды слишком редкие или непонятные, они не будут побуждать к действию.

Социальные аспекты геймификации сильно влияют на вовлечённость. Таблицы лидеров и групповая динамика усиливают соревновательные и кооперативные мотивы. Однако нужно учитывать, что разные пользователи по-разному реагируют на конкуренцию. Для одних людей открытое соревнование служит мотиватором. Для других оно, наоборот, снижает желание участвовать.

Поэтому полезно будет предусмотреть настройки приватности, а также механики, ориентированные на личный прогресс, а не только на сравнение с другими участниками.

Оценка эффективности геймификации должна опираться на набор показателей, связанных как с вовлечённостью, так и с учебными результатами. Ключ-

чевыми показателями являются: частота и длительность сессий, количество повторных обращений к платформе, доля завершённых викторин, динамика среднего балла, сохранение знаний с течением времени.

Анализ этих данных позволяет проверить, какие игровые механики дают наибольший эффект, и при необходимости скорректировать систему наград с учётом выявленных моделей поведения.

1.3. Анализ существующих решений

Изучение действующих платформ для проведения викторин позволяет выделить общие подходы к организации интерфейса, игровым механикам и сбору данных, а также обнаружить типичные недостатки, которые обосновывают создание новой системы. Далее рассмотрены наиболее популярные сервисы с точки зрения их возможностей, технической реализации, преимуществ и ограничений.

Kahoot! представляет собой инструмент для синхронных игр в реальном времени. Его интерфейс отличается простотой и направлен на удержание внимания: ведущий запускает игру, участники подключаются по коду и отвечают на вопросы мгновенно. Игровые элементы включают очки за верные и быстрые ответы, а также таблицу лидеров в рамках одной сессии. Основные достоинства – лёгкость начала работы, яркое визуальное оформление и высокая вовлечённость при очных или дистанционных занятиях. Недостатки: слабая аналитика долгосрочных результатов, ограниченная поддержка сложных типов вопросов и необходимость одновременного участия всех игроков для сохранения динамики.

Quizizz поддерживает как синхронный, так и асинхронный режимы. Платформа предоставляет домашние задания, автоматическое оценивание и более подробные отчёты об успеваемости по сравнению с сервисами, ориентированными только на живые игры. Игровые механики включают очки, мемы, аватары и таблицы лидеров. Quizizz даёт больше свободы в настройке порядка прохождения и распределении материалов, однако возможности изменения логики начисления наград и сложной системы достижений остаются ограниченными. Для задач строгого контроля знаний и детального анализа данных это может быть недостаточно.

Moodle (модуль Quiz) ориентирован на профессиональное использование в учебных заведениях. Ключевые преимущества – глубокая интеграция с системой управления обучением (LMS), разнообразные типы вопросов, гибкие механизмы оценки и расширение функционала через плагины. Moodle предоставляет подробную статистику по вопросам и пользователям, а также удобные средства контроля. Однако интерфейс часто считают громоздким, а настройка и администрирование требуют много времени. Игровые механики в базовой версии почти отсутствуют и добавляются отдельными расширениями.

Socrative и аналогичные системы опроса в классе предназначены для быстрого сбора ответов и простоты использования преподавателем. Они удобны для оперативной проверки усвоения материала, но не рассчитаны на накопление достижений в долгосрочной перспективе или создание разветвлённой системы уровней. Mentimeter больше нацелен на интерактивные презентации и опросы; его функционал для оценки знаний и игровых механик ограничен.

Google Forms и аналогичные универсальные формы представляют собой простой способ создания тестов и опросов. Их достоинства – общедоступность, простота и интеграция с сервисами Google. Однако они почти не приспособлены для игр в реальном времени, не поддерживают динамические таблицы лидеров и не содержат встроенных игровых элементов. Аналитика сводится к базовым отчётам и требует дополнительной обработки.

С технической стороны многие коммерческие продукты используют веб-сокеты или собственные сервисы реального времени для синхронных сессий, облачные хранилища для медиафайлов и внешние системы аналитики. Платность и ограниченная открытость API в ряде решений создают препятствия для интеграции с LMS и корпоративными системами. Вопросы конфиденциальности и хранения персональных данных также различаются: образовательные платформы часто предусматривают механизмы соблюдения местного законодательства, тогда как у развлекательных сервисов политика обработки данных может быть менее строгой.

Выявленные в обзоре недостатки определяют рациональные направления для разработки нового приложения. Во-первых, отсутствует гибкая и понятная система игровых механик, которая объединяла бы краткосрочные стимулы (очки, бонусы за скорость, таблицы лидеров) и долгосрочные (уровни, достижения, ежедневные серии). Во-вторых, многие решения предлагают либо мощную

аналитику без удобной геймификации, либо сильную вовлечённость без глубоких средств оценки и отслеживания прогресса. В-третьих, ограниченный набор типов вопросов, слабая персонализация сложности и нехватка адаптивных сценариев затрудняют применение платформ там, где важна индивидуальная траектория обучения. Наконец, проблемы интеграции – отсутствие открытых API и сложность подключения к LMS или единой системе входа – усложняют внедрение решений в образовательную инфраструктуру.

1.4. Функциональные требования

Функциональные требования описывают поведение системы при взаимодействии с пользователем. Система должна поддерживать регистрацию и аутентификацию, как через электронную почту, так и внешних провайдеров (OAuth). Необходимо реализовать ролевую модель с разграничением доступа: администратор, автор вопросов, модератор и участник. Приложение должно предоставлять средства для создания, изменения и группировки викторин, поддерживать несколько типов вопросов (выбор одного варианта, выбор нескольких, короткий ответ, сопоставление, установление порядка) и позволять добавлять к вопросам медиафайлы. Также требуется реализовать настройку параметров сессии: выбор режима (синхронный или асинхронный), установка таймеров, весовых коэффициентов вопросов и механизмов случайной выборки.

Требования геймификации включают начисление очков за правильные ответы, бонусов за быстрое реагирование, учёт серий верных ответов, введение уровней пользователей, достижений и ежедневных серий. Правила расчёта баллов и условия получения достижений должны быть прозрачными и настраиваемыми через интерфейс администратора или автора викторины. В синхронных сессиях система должна отображать таблицу лидеров с обновлением позиций в реальном времени. Для асинхронного режима необходима корректная запись попыток и формирование отчётов по результатам.

Аналитическая подсистема должна обеспечивать хранение и обобщение данных о попытках пользователей, формировать отчёты по отдельным вопросам и по общей статистике викторин. Требуется возможность получать метрики вовлечённости и успеваемости: количество завершённых сессий, средний балл, динамику серий, удержание пользователей и распределение ошибок по вопро-

сам. Должен быть предусмотрен экспорт данных в стандартных форматах для последующей обработки.

1.5. Нефункциональные требования

Нефункциональные требования охватывают производительность, масштабируемость, доступность, безопасность, конфиденциальность и удобство использования.

Производительность и масштабируемость. Система должна выдерживать одновременное участие не менее 100 активных пользователей в синхронной сессии для базовой версии (MVP) и быть масштабируемой до 1000 и более пользователей при последующем развитии. Время отклика API для типовых операций при нормальной нагрузке не должно превышать нескольких сотен миллисекунд. Обновление таблицы лидеров в ходе живой сессии должно происходить с задержкой не более одной секунды в стандартных условиях.

Доступность. Система должна обеспечивать непрерывность работы с целевым уровнем доступности, сопоставимым с показателем SLA 99.5% в производственной среде.

Безопасность. Требуется защита веб-интерфейса от распространённых уязвимостей (XSS, CSRF, SQL-инъекции), надёжное хранение паролей с применением современных алгоритмов хеширования и обязательное шифрование каналов связи (TLS). Необходимо соблюдение требований к защите персональных данных, включая возможность удаления и обезличивания записей по запросу пользователя в соответствии с действующими нормативными актами. Для социальных функций следует предусмотреть настройки приватности и возможность ограничения видимости результатов по выбору пользователя.

Доступность. Система должна обладать адаптивным дизайном для мобильных устройств и настольных компьютеров, интуитивно понятным интерфейсом для каждого типа пользователя и соответствовать базовым требованиям доступности (WCAG) в пределах, допустимых объёмом проекта. Поддержка современных браузеров и их версий должна быть явно зафиксирована в спецификации на этапе разработки и тестирования.

1.6. Выбор и обоснование технологического стека

Выбор технологического стека обусловлен функциональными и нефункциональными требованиями проекта: необходимостью быстрой разработки минимально жизнеспособного продукта (MVP), надёжного хранения структурированных данных, а также возможностью масштабируемого развёртывания с поддержкой работы в реальном времени и фоновой обработки задач.

Основной фреймворк: Django. Данное решение обосновано концепцией «всё необходимое включено в комплект». Встроенные механизмы аутентификации, управления правами доступа, защиты от распространённых веб-уязвимостей и готовая административная панель существенно сокращают объём типовой работы. Это позволяет разработчику сосредоточиться на предметной области – логике проведения викторин и игровых механиках. Фреймворк Django REST Framework обеспечивает удобную и расширяемую реализацию API для одностраничных приложений (SPA) или мобильных клиентов. Встроенная объектно-реляционная модель (ORM) и система миграций упрощают управление схемой данных в процессе развития проекта.

PostgreSQL выбран в качестве основной СУБД исходя из требований к надёжности, целостности транзакций и аналитическим возможностям. Поддержка сложных запросов, различных типов индексов, расширений и типизированных полей (включая тип JSONB) делает PostgreSQL подходящим для хранения как классических реляционных сущностей (вопросы, попытки, результаты), так и гибких аналитических данных. Механизмы транзакций и контроля параллельного доступа важны для корректного подсчёта очков и учёта серий ответов в условиях конкурентных обновлений. Кроме того, PostgreSQL предоставляет инструменты для репликации и резервного копирования, необходимые для обеспечения доступности и сохранности данных.

Контейнеризация: Docker. Docker включён в состав стека как средство развёртывания и стандартизации окружения. Контейнеризация обеспечивает воспроизводимость среды разработки и продукционной среды, изоляцию зависимостей и предсказуемость поведения приложения при переносе между различными машинами. Для локальной разработки и непрерывной интеграции (CI) целесообразно использовать Docker Compose, позволяя описать в одном файле все необходимые сервисы: веб-приложение (Gunicorn / uvicorn), базу данных, кэш

и брокер сообщений (Redis или RabbitMQ), фоновый обработчик задач (Celery), обратный прокси-сервер (Nginx) и сервисы мониторинга. Для продуктивной среды рекомендуются оркестраторы (Kubernetes или Docker Swarm) либо облачные платформенные сервисы, обеспечивающие автоматическое масштабирование, управление секретами и перезапуск контейнеров при сбоях. Использование многоступенчатой сборки (multi-stage) и минимальных базовых образов позволяет уменьшить размер образов и ускорить развёртывание.

Для обеспечения производительности возможно применить кэширование на нескольких уровнях: HTTP-кэширование, кэш в Redis для частых запросов и использование PgBouncer для пула соединений с базой данных. Для аналитики и агрегатов логично хранить детальные данные в PostgreSQL, а для быстрых выборок использовать материализованные представления или отдельные агрегированные таблицы, обновляемые через фоновые задачи.

Резервное копирование базы данных рекомендуется организовать с помощью `pg_dump` или `pg_basebackup` с хранением копий в удалённом объектном хранилище и регулярной проверкой восстановления. Логирование и мониторинг реализуются через связку Sentry для отслеживания ошибок приложения и Prometheus с Grafana для сбора метрик производительности. Централизованное хранение логов может быть организовано с использованием стека ELK или специализированного облачного сервиса.

Альтернативные решения (Node.js с Socket.io, Firebase, Ruby on Rails) обладают собственными преимуществами, однако в контексте образовательного проекта Django даёт выигрыш в скорости разработки, наличии готовых инструментов безопасности и встроенного административного интерфейса. Docker делает выбранный стек переносимым и готовым к автоматизации развёртывания.

Глава 2. Практическая часть

2.1. Проектирование базы данных

На диаграмме (рис. 2.1) представлены 10 сущностей, разделённых на четыре модуля системы – accounts, quiz, session, leaderboard.

Центральной сущностью всей системы является таблица `accounts_user`. Она расширяет стандартную модель пользователя Django и хранит не только аутентификационные данные (логин, адрес электронной почты, пароль), но и игровой профиль пользователя: накопленные очки, текущий уровень, количество дней подряд активности (серия) и статистику ответов. Такой подход позволяет обойтись без отдельной таблицы профиля.

Таблица `accounts_achievement` содержит каталог достижений. Каждое достижение описывается категорией, иконкой, наградой в очках и пороговым значением, необходимым для разблокировки. Связь между пользователем и достижениями реализована через промежуточную таблицу `accounts_userachievement`, которая фиксирует момент получения каждой награды. Отношение «многие ко многим» реализуется именно через эту таблицу.

Таблица `quiz_category` служит для группировки викторин по тематике. Каждая викторина (`quiz_quiz`) принадлежит ровно одной категории и одному создателю; связи «многие к одному» установлены с таблицами `quiz_category` и `accounts_user` соответственно.

Викторина состоит из вопросов (`quiz_question`). Каждый вопрос содержит несколько вариантов ответа (`quiz_answer`), причём ровно один из них помечен как правильный. Таким образом, реализована классическая двухуровневая иерархия «один ко многим»: викторина → вопросы → ответы.

Когда пользователь начинает викторину, создаётся запись в `quiz_quizsession`, которая связывает пользователя и конкретную викторину. В этой записи накапливаются итоговые показатели: общий счёт, количество правильных ответов, максимальная серия (стрик) подряд верных ответов и затраченное время.

Каждый отдельный ответ пользователя фиксируется в `quiz_useranswer`. Данная таблица является наиболее связанной в схеме: она одновременно ссылается на сессию, вопрос и выбранный вариант ответа, а также хранит флаг правильности и количество заработанных очков. Такая структура позволяет восстановить

полную картину любого прохождения викторины.

Таблица `leaderboard_entry` агрегирует позицию каждого пользователя в таблице лидеров. Поле `period` позволяет хранить рейтинги за разные временные интервалы (за всё время, за месяц, за неделю) в одной таблице. Отношение с пользователем — «многие к одному».

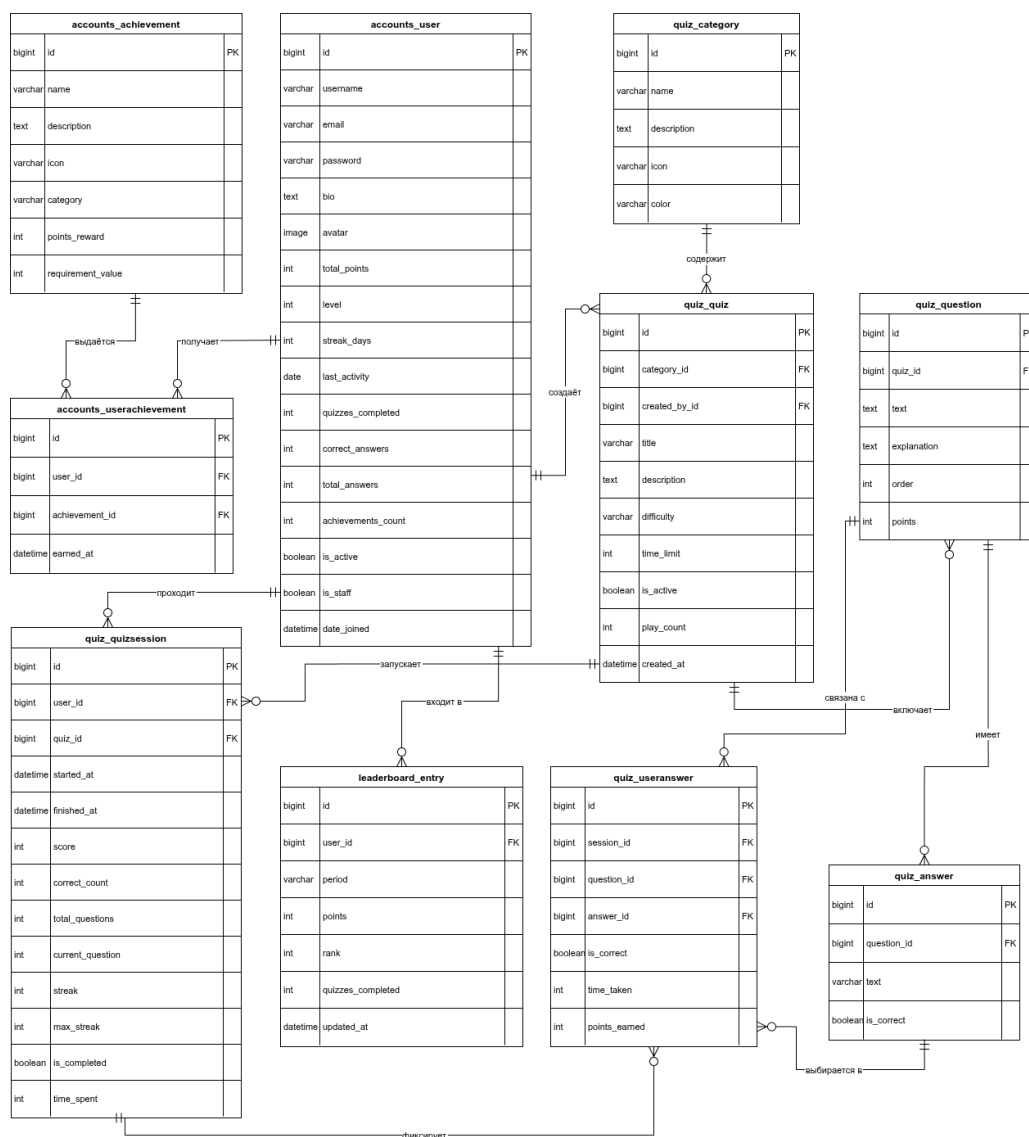


Рисунок 2.1 – ER-диаграмма разрабатываемой системы

2.2. Структура проекта

Проект организован в соответствии со стандартными принципами Django — каждый функциональный блок вынесен в отдельное приложение:

Листинг 1 – Структура веб-приложения

```
quiz_app/
├── config/                # Конфигурация проекта
│   ├── settings.py       # Настройки
│   ├── urls.py           # Корневой маршрутизатор
│   └── wsgi.py           # Точка входа WSGI-сервера
│
├── accounts/             # Модуль пользователей
│   ├── models.py         # Модели модуля пользователей
│   ├── views.py          # Регистрация, вход, профиль
│   ├── forms.py          # Формы
│   ├── urls.py           # Маршруты /accounts/
│   └── admin.py          # Управления пользователями
│
├── quiz/                 # Модуль викторин
│   ├── models.py         # Модели модуля викторин
│   ├── views.py          # Отображение элементов
│   ├── urls.py           # Маршруты /quizzes/, /play/
│   └── admin.py          # Управление викторинами
│
├── leaderboard/          # Модуль рейтинга
│   ├── models.py         # LeaderboardEntry
│   ├── views.py          # Таблица лидеров
│   └── urls.py           # Маршрут /leaderboard/
│
├── templates/            # HTML-шаблоны
│   ├── base.html         # Базовый макет
│   ├── accounts/
│   │   ├── login.html
│   │   ├── register.html
│   │   ├── profile.html
│   │   ├── edit_profile.html
│   │   └── public_profile.html
```



```

|   └─ quiz/
|   │   └─ home.html    # Главная
|   │   └─ list.html    # Список с фильтрами
|   │   └─ detail.html  # Карточка викторины
|   │   └─ play.html    # Игровой экран
|   │   └─ result.html  # Результаты
|   └─ leaderboard/
|       └─ leaderboard.html
|
└─ static/
    └─ css/main.css      # Все стили
    └─ js/main.js        # Вспомогательные скрипты
|
└─ seed_data.py          # Демо-версия БД
└─ requirements.txt       # Зависимости Python
└─ Dockerfile            # Образ контейнера
└─ docker-compose.yml    # Оркестрация web + db
└─ .env.example          # Пример переменных окружения

```

Модуль `config` содержит глобальные настройки проекта. В `settings.py` вынесены параметры подключения к PostgreSQL через переменные окружения, пути к статическим и медиафайлам, а также константы геймификации – очки за правильный ответ, бонус за скорость ответа и бонус за серию верных ответов.

Модуль `accounts` отвечает за весь жизненный цикл пользователя: регистрацию, аутентификацию, хранение игрового профиля. Пользовательская модель `User` наследует `AbstractUser` и расширена полями геймификации. Логика начисления уровней инкапсулирована в методах модели (`update_level`, `level_progress`), что избавляет представления от избыточных вычислений.

Модуль `quiz` представляет собой ядро системы. Представления реализуют полный цикл прохождения викторины:

- создание сессии;
- страничный показ вопросов с таймером;
- AJAX-обработка ответа (функция `submit_answer_view`);

- финализация сессии с обновлением статистики;
- экран с результатами.

Проверка достижений (`_check_achievements`) и обновление статистики пользователя (`_update_user_stats`) вызываются автоматически при завершении сессии.

Модуль `leaderboard` намеренно оставлен лёгким: он не дублирует данные, а агрегирует их непосредственно из таблицы `accounts_user`, выстраивая рейтинг по полю `total_points`.

Шаблоны построены на механизме наследования от `base.html`, который обеспечивает единый макет, навигацию и вывод системных сообщений. Игровой экран (`play.html`) содержит встроенный JavaScript, реализующий:

- таймер обратного отсчёта;
- отправку ответа через Fetch API без перезагрузки страницы;
- анимацию начисленных очков.

Скрипт `seed_data.py` при первом запуске выполняет следующие действия:

- создание суперпользователя;
- создание демонстрационного аккаунта;
- заполнение набора достижений;
- создание категорий;
- создание трёх готовых викторин с вопросами.

Благодаря этому система сразу готова к демонстрации без необходимости ручного ввода данных через административную панель.

2.3. Описание реализованного функционала

Аутентификация построена на основе модуля `django.contrib.auth` с расширением через пользовательскую модель `User`, которая унаследована от `AbstractUser`. В системе реализованы следующие возможности:

- Для регистрации используется форма `RegisterForm` построенная на основе `UserCreationForm` с проверкой уникальности логина, корректности адреса электронной почты и надёжности пароля. При успешной регистрации пользователь автоматически авторизуется в системе;
- Для входа и выхода пользователя применяется форма `LoginForm` на ос-

нове `AuthenticationForm`. Выход из системы завершает сессию и перенаправляет пользователя на страницу входа;

- Все страницы, требующие авторизации, специально были защищены декоратором `@login_required` с автоматическим перенаправлением неавторизованных пользователей на форму входа;
- Пользователь имеет возможность изменить имя, адрес электронной почты, биографию и загрузить аватар;
- Страница любого пользователя является публичной, доступна по адресу `/accounts/profile/<username>/` и отображает статистику и достижения без возможности редактирования.

В модуле викторин реализован полный цикл прохождения викторины – от выбора до просмотра результатов.

Каталог викторин поддерживает фильтрацию одновременно по категории и уровню сложности (лёгкий, средний, сложный). Карточка каждой викторины отображает категорию, сложность, количество вопросов, лимит времени и число прохождений.

Запуск викторины создаёт объект `QuizSession`, который фиксирует момент начала и привязывает прохождение к конкретному пользователю. Счётчик прохождений викторины (`play_count`) увеличивается при каждом запуске.

Игровой экран реализован без перезагрузки страницы:

- Таймер обратного отсчёта отрисован средствами JavaScript и визуализирован в виде прогресс-бара, который меняет цвет с зелёного на красный при остатке времени менее 5 секунд.
- При истечении времени ответ засчитывается автоматически как пропущенный.
- Выбранный вариант отправляется на сервер через Fetch API (POST-запрос на `/play/<id>/submit/`). В ответ возвращается JSON-объект с результатом, правильным ответом и пояснением, которые отображаются немедленно без перехода на новую страницу.

Экран результатов показывает итоговую оценку по пятибалльной шкале; набранные очки, точность ответов, количество правильных ответов и максимальную серию (стрик); полный разбор всех вопросов с указанием выбранного и правильного ответов, а также пояснений (там, где они заданы).

Начисление очков происходит по следующей схеме (таблица 1):

Таблица 1 – Соответствие события и награды в виде очков

Событие	Очки
Правильный ответ	+10
Быстрый ответ (менее 5 секунд)	+3
Серия (каждые 3 правильных ответа подряд)	+5
Получение достижения	от +50 до +250

Система включает 7 уровней с названиями от «Новичок» до «Легенда». Переход на следующий уровень происходит автоматически при достижении порогового значения очков. На странице профиля отображается прогресс-бар, показывающий продвижение к следующему уровню.

Стрики отслеживаются в двух измерениях:

- серия правильных ответов внутри одной викторины (влияет на бонусные очки);
- серия дней подряд с активностью (хранится в профиле пользователя).

Реализовано 8 достижений четырёх категорий: за количество пройденных викторин, за дневные серии, за накопленные очки и за точность ответов. Проверка условий разблокировки выполняется автоматически после каждого завершённого прохождения. При получении нового достижения на экране результатов появляется анимированное всплывающее окно с названием, описанием и размером награды.

На странице профиля агрегируется полная статистика пользователя:

- суммарные очки;
- количество пройденных викторин;
- процент правильных ответов;
- текущая дневная серия.

Также отображаются все полученные достижения и история последних пяти прохождений с указанием даты, набранных очков и точности ответов.

Глобальный рейтинг формируется на основе суммарных очков всех пользователей. Первые три места отмечены медалями. Для авторизованного пользователя подсвечивается его строка и отображается текущая позиция в рейтинге. По каждому участнику видны уровень, количество пройденных викторин и точность ответов.

Стандартная панель Django Admin расширена для удобного управления контентом. Администратор имеет возможность управлять пользователями, категориями, достижениями и просматривать историю всех сессий.

Проект контейнеризован с помощью Docker Compose и включает два сервиса:

- db – база данных на основе PostgreSQL 15;
- web – приложение на Python 3.12.10.

Механизм `healthcheck` гарантирует, что Django не запускается до полной готовности базы данных. Миграции применяются автоматически при запуске контейнера. Скрипт `seed_data.py` заполняет базу данных демонстрационными данными за один запуск.

Заключение

Текст заключения.

Список использованных источников

1. Author Book3
2. Author Book1
3. Author Book2

Приложения

Текст приложений.